

Software Developer — Coding Assessment Guidance

Overview

Thank you for your interest in the Software Developer position. This assignment is designed to evaluate your practical skills in **C#** and **SQL**, along with your ability to write clean, maintainable code and communicate your work clearly.

- **Estimated time:** 6–8 hours
- **Deadline:** Please submit within **5 business days** of receiving this document
- **Submission:** Public or private GitHub repository (instructions at the end)

*Note: Code quality, structure, and clarity of thought matter just as much as feature coverage. If you run out of time, document what's left and how you would approach it.

Scenario: Library Management System

You are building a small app for a community library. Librarians need to manage **books**, **members**, and **loans** (a member borrowing a book). Your task is split into two parts: a C# application and a set of SQL tasks against a shared schema.

Part A — C# Implementation (60% of evaluation)

Build a **ASP .NET Core Web Application (.Net 8 and above)** that exposes the following functionality.

Required Features

1. **Books**
 - Add a new book (`Title`, `Author`, `ISBN`, `PublishedYear`, `TotalCopies`)
 - List all books with optional filtering by author or title (case-insensitive, partial match)
 - Retrieve a single book by ID, including how many copies are currently available
2. **Members (authenticated via SSO)**
 - Members **do not register manually**. Instead, they sign in through an external Identity Provider using **OpenID Connect (OIDC)**.
 - On a member's first successful sign-in, your application must **provision** a `Member` record by reading claims from the ID token (at minimum: a stable subject identifier, `FullName`, and `Email`). Set `JoinedDate` to the provisioning timestamp.

- On subsequent sign-ins, look up the existing member by the SSO subject identifier.
 - Expose a `GET /me` endpoint that returns the currently authenticated member's profile.
 - Member-related endpoints must derive the member's identity from the access token, **not** from a value supplied in the request body or URL.
3. **Loans**
- Borrow a book: the **authenticated member** borrows one copy of a specific book
 - Reject if no copies are available
 - Reject if the member already has **3 or more** active (unreturned) loans
 - Return a book: mark a loan as returned and record the return date. A member can only return their own loans.
 - List active loans for the **currently authenticated member** (`GET /me/loans`)

Technical Requirements

- **Language: C# only.** Do not introduce other languages (e.g., F#, VB.NET, TypeScript). Auto-generated migration code is fine.
- **Framework: ASP.NET Core Web Application** on .NET 8 and above. Console apps or other project types will not be accepted.
- **IDE: Visual Studio 2022** (the free **Community Edition** is acceptable). Please do not use Rider or VS Code so that your solution opens cleanly on our reviewers' machines.
- **Database: Microsoft SQL Server** only. Use any **free** edition or tool, such as:
 - **SQL Server 2022 Express** or **Developer Edition** (free)
 - **LocalDB** (bundled with Visual Studio)
 - **SQL Server in Docker** (mcr.microsoft.com/mssql/server, free Developer image)
 - Management tooling: **SQL Server Management Studio (SSMS)** or **Azure Data Studio** — both free
- **Data access:** Entity Framework Core OR Dapper — your choice, justify it briefly in the README
- **Authentication: SSO via OpenID Connect** (see section below)
- **Validation:** Use model validation (FluentValidation or DataAnnotations)
- **Error handling:** Return appropriate HTTP status codes with meaningful error messages
- **Testing:** At least **3 unit tests** for non-trivial business logic (e.g., the loan limit rule and the SSO provisioning flow). xUnit, NUnit, or MSTest are all fine.
- **Logging:** Basic structured logging (built-in `ILogger` is acceptable)

Your README must state the **connection string format** you used and which SQL Server edition/tool a reviewer should install to run the project locally.

Authentication Requirements (SSO)

All member-facing endpoints (`/me`, `/me/loans`, `borrow`, `return`) must be protected and require a valid token issued by an external Identity Provider.

- Use an **OpenID Connect (OIDC)** provider. You may choose any of the following:
 - **Microsoft Entra ID** (formerly Azure AD)
 - **Google** Identity Platform
 - any other standards-compliant OIDC provider
- Use the standard ASP.NET Core authentication middleware (`Microsoft.AspNetCore.Authentication.JwtBearer` or `OpenIdConnect`). **Do not roll your own token validation.**
- Validate **issuer**, **audience**, **signature**, and **expiry** on every request.
- Protected endpoints must return **401 Unauthorized** for missing/invalid tokens and **403 Forbidden** when a member attempts an action on another member's data.
- **Book management endpoints** (create book, list books) may either be public, require any authenticated user, or require a `librarian` role — your choice, documented in the README.
- **Do not commit client secrets or signing keys.** Use `appsettings.json` with placeholder values and document any required environment variables in the README.

Your README must explain how a reviewer can either (a) point the app at the same provider you used, or (b) run a local Keycloak/mock OIDC container to obtain test tokens.

Part B — SQL (40% of evaluation)

Using the same domain as Part A, complete the following SQL tasks. Submit your answers as `.sql` files inside a `/sql` folder.

All SQL must be written in **T-SQL** (Microsoft SQL Server dialect) and verified to run on SQL Server 2019 or later. Each file should be runnable as-is in SSMS or Azure Data Studio.

B1. Schema

Provide a `schema.sql` file containing your `CREATE TABLE` statements for `Books`, `Members`, and `Loans`. Include primary keys, foreign keys, indexes, and any constraints you consider necessary. Briefly justify your indexing choices as SQL comments.

B2. Seed Data

Provide a `seed.sql` file with sample data: at least 10 books, 5 members, and 10 loans (a mix of returned and active).

B3. Queries

Write the following queries in a `queries.sql` file. Each query should be preceded by a comment stating the question number.

1. List the **top 5 most-borrowed books** of all time, showing title, author, and total times borrowed.
 2. Find all members who have **at least one overdue loan** (active loan older than 14 days). Show the member's name, email, and number of overdue loans.
 3. For each month of the **last 12 months**, return the total number of loans created. Include months with zero loans.
 4. Find books that have **never been borrowed**.
 5. Identify the member with the **longest single loan duration** (returned loans only). Show member name, book title, and number of days borrowed.
-

Evaluation Criteria

Your submission will be evaluated on:

Area	What we look for
Correctness	Features work as specified; business rules enforced
Code quality	Readable, well-named, appropriately abstracted; no dead code
Architecture	Sensible separation of concerns; testable design
SQL proficiency	Correct queries, sound schema design, awareness of performance
Testing	Meaningful tests covering business rules, not just happy paths
Documentation	Clear README; reviewer can run the project in under 10 minutes
Git hygiene	Meaningful commits; no committed secrets, binaries, or <code>bin/obj</code> folders

Submitting via GitHub

Please share your completed assignment as a Git repository on GitHub. Follow these steps:

1. Create a GitHub account

If you don't already have one, sign up at github.com.

2. Create a new repository

- Go to github.com/new
- Name it `library-assessment-<yourname>` (e.g., `library-assessment-jane-doe`)
- Set visibility to **Private** (recommended) or Public — your choice
- Do **not** initialize with a README — you'll push your own

3. Initialize the project locally and push

From your project's root folder, run:

```
bash
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/<your-username>/library-assessment-<yourname>.git
git push -u origin main
```

4. Include a clear `README.md`

Your README should cover:

- A short description of the project
- **Setup instructions** (prerequisites, how to install dependencies)
- **How to run** the application
- **How to run the tests**
- **How to apply the database schema and seed data**
- Any **assumptions** or **design decisions** you'd like to call out
- Anything left unfinished and how you would complete it

5. Add a `.gitignore`

Use the standard Visual Studio / .NET `.gitignore` to avoid committing build artifacts. You can generate one at gitignore.io/api/visualstudio,dotnetcore.

6. Grant access (if your repo is private)

Add the following GitHub usernames as collaborators with **Read** access:

- <reviewer-username-1>
- <reviewer-username-2>

To add collaborators: **Repository** → **Settings** → **Collaborators** → **Add people**.

7. Send us the link

- The repository URL
 - The commit hash of your final submission (run `git rev-parse HEAD`)
 - A short note on anything you'd like us to know before reviewing
-

If anything in this assignment is unclear, please reach out to us through email
Iskandar@uniten.edu.my

Good luck, and we look forward to reviewing your work!